

SMART FRAUD DETECTION PIPELINE

Using PySpark | Spark SQL | Azure Databricks | Delta Lake

Medallion Architecture

Dataset: Sample Superstore Dataset

Author: Snehal A. Bhosale

College: Sanjivani College of Engineering, Kopergaon – 423603

E-mail: snehalbhosale1807@gmail.com

CEI ID: CT_CSI_DE_1177

Project Title

"Smart Fraud Detection Pipeline using PySpark, Spark SQL, Azure Databricks and Delta Lake"

1. Abstract

This report presents the design and implementation of an end-to-end fraud detection data pipeline built using PySpark, Spark SQL, Azure Databricks, and Delta Lake. The pipeline processes raw financial data—comprising account records, transaction logs, and a fraud watchlist—through a structured Medallion Architecture consisting of Bronze, Silver, and Gold layers. Data is ingested, cleaned, validated, enriched, and matched against fraud indicators to classify each transaction as Fraud or Normal. The second phase of the project migrates the pipeline to Azure Databricks and introduces Delta Lake for reliable cloud-based storage, ACID-compliant table management, and incremental processing through the MERGE operation. The result is a scalable, modular, and production-grade fraud detection solution demonstrating core data engineering competencies.

2. Objective

The primary objective of this project is to design and implement an end-to-end fraud detection data pipeline that demonstrates practical competency in modern data engineering. The specific objectives are as follows:

- Process and clean raw financial transaction data from multiple CSV sources.
- Apply the Medallion Architecture (Bronze, Silver, Gold) for structured data flow.
- Detect fraudulent transactions by matching against a fraud watchlist.
- Generate meaningful business insights using Spark SQL analytical queries.
- Extend the pipeline to Azure Databricks for cloud-based distributed processing.
- Store all data layers as Delta Lake tables supporting ACID transactions.

- Demonstrate incremental data processing using the Delta Lake MERGE operation.

3. Problem Statement

Financial institutions process millions of transactions daily. Within this volume, fraudulent activity is difficult to identify manually. Raw transaction data typically contains numerous quality issues including missing values, duplicate records, invalid date formats, non-numeric amounts, and orphan account references that do not exist in the accounts master. Additionally, traditional data lake storage formats such as CSV and plain Parquet do not support reliable update or delete operations. Any modification to existing data requires rewriting entire files, which is slow, error-prone, and unsuitable for production pipelines where transaction data is continuously arriving.

This project addresses these challenges by building a structured pipeline that:

- Identifies and resolves all data quality issues in raw transaction data.
- Classifies each transaction as Fraud or Normal using a fraud watchlist.
- Stores all processed layers in Delta format to support ACID-compliant operations.
- Uses the Delta MERGE operation to handle incremental data without duplication.

4. Technology Stack

Technology	Purpose
Python	Core programming and scripting language
PySpark	Distributed data processing framework
Spark SQL	SQL-based analytical queries on distributed datasets
Jupyter Notebook	Initial development, exploration, and analysis
Azure Databricks	Cloud platform for scalable Spark execution
Delta Lake	Reliable storage layer with ACID transactions and MERGE support
Pandas	Result inspection and lightweight data manipulation
Matplotlib	Data visualization and chart generation

5. Dataset Description

The project uses three realistic financial datasets that together simulate a real-world banking transaction environment.

5.1 Accounts

Column	Description
account_id	Unique identifier for each bank account
customer_name	Full name of the account holder
account_type	Account category: Savings, Current, or Salary
credit_limit	Maximum credit allowed on the account
branch	Bank branch where the account is registered

5.2 Transactions

Column	Description
txn_id	Unique identifier for each transaction
account_id	Account associated with this transaction
txn_date	Date on which the transaction occurred
amount	Transaction amount in currency units
merchant	Merchant or location where the transaction took place

5.3 Fraud Watchlist

Column	Description
account_id	Account flagged for suspected fraudulent activity
fraud_type	Category of fraud detected (e.g., Identity Theft, Card Fraud)
flagged_date	Date on which the account was formally flagged

6. Dataset Quality Issues

The datasets were constructed to include realistic data quality problems that are common in production financial systems. These issues are identified during data exploration and resolved in the Silver layer.

6.1 Accounts

- Duplicate account IDs
- Missing account ID values

- Extra leading and trailing whitespace in string fields
- Missing customer name entries
- Missing branch values
- Negative credit limit values

6.2 Transactions

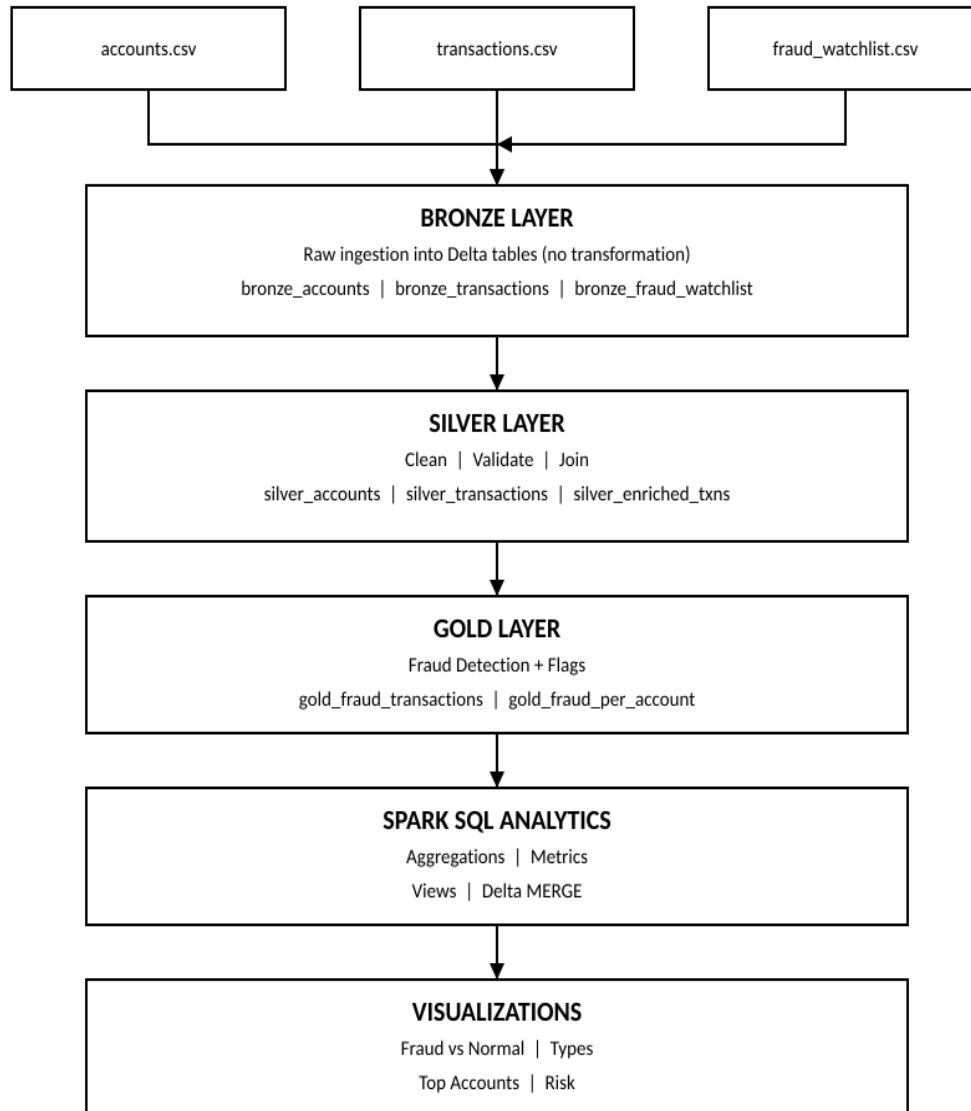
- Missing account ID references
- Missing transaction date values
- Invalid and inconsistent date formats
- Missing and non-numeric amount values
- Negative transaction amounts
- Duplicate transaction records
- Orphan account IDs not present in the accounts dataset

6.3 Fraud Watchlist

- Duplicate account entries
- Missing fraud type values
- Invalid date formats in `flagged_date`
- Extra whitespace in string fields
- Orphan account IDs not present in accounts dataset

7. System Architecture

The pipeline follows the Medallion Architecture, a standard data engineering pattern that organises data into three distinct quality layers: Bronze, Silver, and Gold. Each layer serves a specific purpose and feeds into the next.



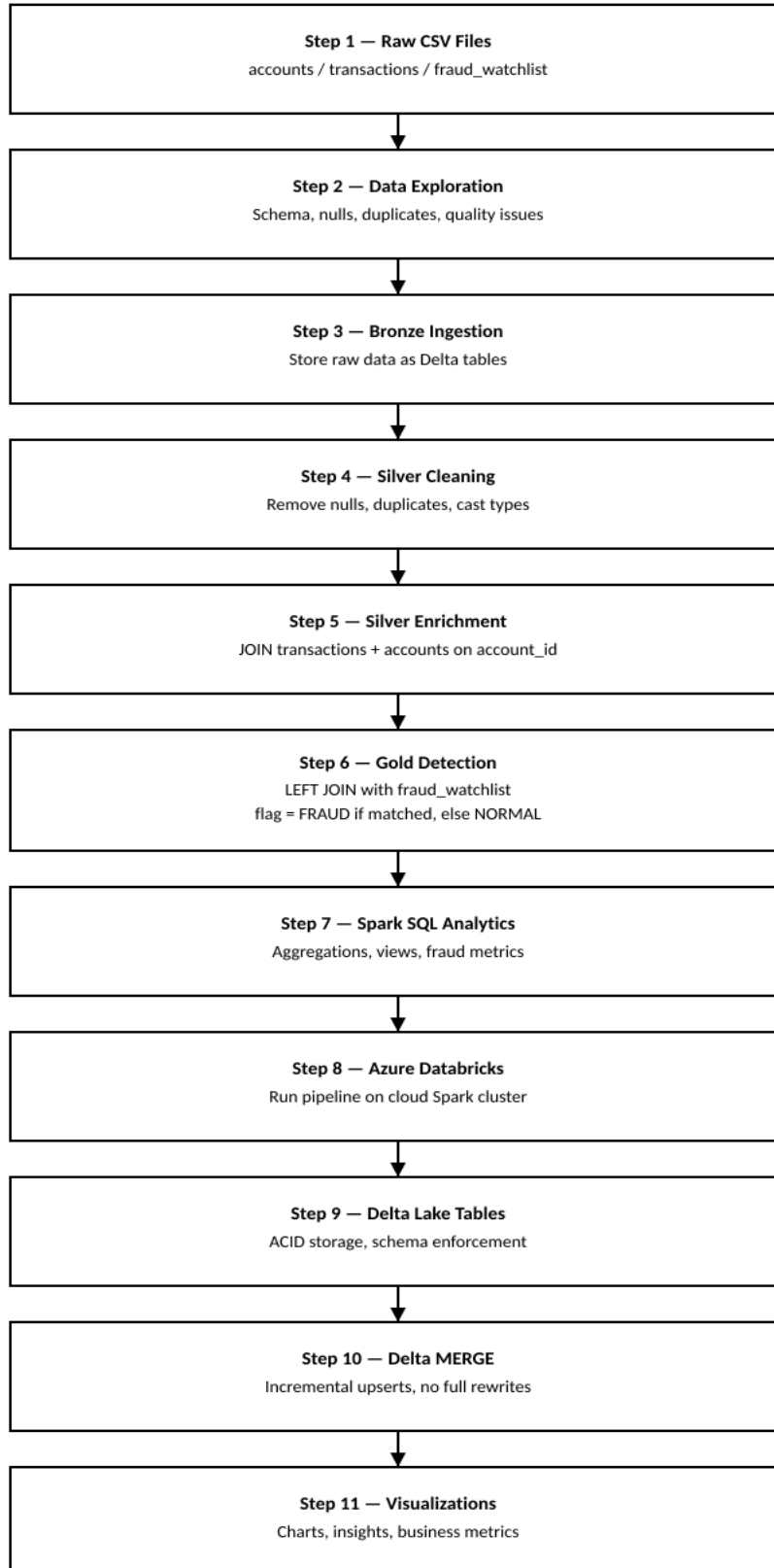
7.1 Layer Summary

Layer	Purpose / Tables Created
Bronze	Raw data ingestion into Delta tables with no transformation — <code>bronze_accounts</code> , <code>bronze_transactions</code> , <code>bronze_fraud_watchlist</code>
Silver	Data cleaning, validation, type casting, deduplication, and enrichment through joins — <code>silver_accounts</code> , <code>silver_transactions</code> , <code>silver_enriched_transactions</code>

Layer	Purpose / Tables Created
Gold	Fraud detection logic, transaction classification, and account-level aggregations — gold_fraud_transactions, gold_fraud_per_account

8. End-to-End Workflow

The following diagram presents the complete sequential flow of the pipeline from raw data ingestion through to the final visualisations.



9. Medallion Architecture — Detailed Description

9.1 Bronze Layer

The Bronze layer is the entry point of the pipeline. Its sole responsibility is to ingest the three raw CSV files into the Spark environment and persist them as Delta tables without any transformation. This approach preserves the original source data for audit, lineage tracking, and potential reprocessing.

Key operations:

- Read raw CSV files using `spark.read.csv()` with `inferSchema` enabled.
- Write each DataFrame to Delta format using `.write.format("delta").mode("overwrite")`.
- Register each Delta table in the Spark catalog using `CREATE TABLE ... USING DELTA LOCATION`.

Sample code:

```
accounts_df      =      spark.read.csv("accounts.csv",      header=True,
                                     inferSchema=True)
accounts_df.write.format("delta").mode("overwrite")      \
    .saveAsTable("bronze_accounts")
```

9.2 Silver Layer

The Silver layer performs all data cleaning and enrichment operations. Each dataset is processed individually before the final enriched join is created.

Processing steps applied to each dataset:

- `dropDuplicates()` to remove exact duplicate records.
- `dropna()` on critical columns such as `account_id`, `txn_id`, and `amount`.
- `col("amount").cast("double")` to enforce numeric types.
- `to_date()` for standardising date formats.
- `trim()` to remove extra whitespace from string columns.
- Filter to remove negative amounts and invalid credit limits.

After individual cleaning, transactions are joined with accounts:

```
silver_enriched      =      silver_txns.join(silver_accounts,
                                     on="account_id",      how="inner")
silver_enriched.write.format("delta").mode("overwrite")      \
    .saveAsTable("silver_enriched_transactions")
```

9.3 Gold Layer

The Gold layer contains the business-ready fraud detection output. Enriched transactions are matched against the fraud watchlist using a left join on `account_id`. A conditional column `fraud_flag` is derived to classify each transaction.

Fraud detection logic:

- If a transaction's account_id exists in the fraud watchlist: fraud_flag = "fraud"
- If no match is found: fraud_flag = "normal"

Implementation:

```
from pyspark.sql.functions import when, col, lit
gold_df = silver_enriched.join(watchlist, on="account_id",
how="left")
gold_df = gold_df.withColumn("fraud_flag",
    when(col("fraud_type").isNotNull(),
lit("fraud")).otherwise(lit("normal")))
gold_df.write.format("delta").mode("overwrite") \
    .saveAsTable("gold_fraud_transactions")
```

10. Data Quality Validation

Quality checks are applied at both the Silver and Gold layers to prevent invalid records from propagating silently into analytical outputs. The following validation categories are applied:

Validation Type	Method / Layer
Null value detection	filter(col("column").isNull()).count() — Silver, Gold
Duplicate detection	groupBy().count().filter(count > 1) — Silver
Invalid date detection	to_date() returning null — Silver
Negative amount detection	filter(col("amount") < 0) — Silver
Orphan account detection	anti-join against accounts — Silver
Row count validation	assert df.count() > 0 — Gold
Gold null check	assert gold.filter(col("account_id").isNull()).count() == 0 — Gold

Example assertion used in the Gold layer:

```
assert gold_transactions.filter(col("account_id").isNull()).count()
== 0, \
    "Gold layer contains null account_id values"
```

11. Spark SQL Analytics

Spark SQL is used to generate business-level fraud insights directly from the Gold Delta table. Views and queries are created for each key metric.

11.1 Fraud vs Normal Summary

```
SELECT
    COUNT(*) AS transaction_count,
    SUM(amount) AS total_amount
FROM
    gold_fraud_transactions
GROUP BY fraud_flag;
```

11.2 Fraud by Account

```
SELECT
    COUNT(*) AS fraud_transactions,
    SUM(amount) AS fraud_amount
FROM
    gold_fraud_transactions
WHERE
    fraud_flag = 'fraud'
GROUP BY
    account_id, fraud_type
ORDER BY fraud_amount DESC;
```

11.3 Fraud by Merchant

```
SELECT
    merchant, COUNT(*) AS fraud_txns
FROM
    gold_fraud_transactions
WHERE
    fraud_flag = 'fraud'
GROUP BY
    merchant
ORDER BY fraud_txns DESC;
```

12. Azure Databricks Implementation

The second phase of the project migrates the complete pipeline from a local Jupyter environment to Azure Databricks. The same Medallion Architecture is applied using Databricks notebooks and Delta Lake as the storage layer.

12.1 Setup

- An Azure Databricks workspace was created via the Azure portal.
- A Spark cluster was configured with the latest Databricks Runtime (includes Delta Lake by default).
- Source CSV files were uploaded to DBFS at dbfs:/FileStore/fraud/.
- A dedicated database fraud_db was created to hold all Delta tables.

12.2 Delta Table Creation in Databricks

All three pipeline layers are implemented as registered Delta tables within fraud_db:

```
spark.sql("CREATE DATABASE IF NOT EXISTS fraud_db")

# Bronze
df.write.format("delta").mode("overwrite")
    .saveAsTable("fraud_db.bronze_accounts")
```

```

#                                                    Silver
silver_df.write.format("delta").mode("overwrite") \
    .saveAsTable("fraud_db.silver_enriched_transactions")

#                                                    Gold
gold_df.write.format("delta").mode("overwrite") \
    .saveAsTable("fraud_db.gold_fraud_transactions")

```

12.3 Delta Table Metadata and History

Delta Lake maintains a transaction log for every write operation. The following SQL commands expose this metadata:

```

--      View      table      structure      and      storage      details
DESCRIBE      DETAIL      fraud_db.gold_fraud_transactions;

--      View      full      write      history
DESCRIBE HISTORY fraud_db.gold_fraud_transactions;

```

13. Delta Lake MERGE — Incremental Processing

13.1 Why MERGE is Required

Traditional data pipelines handle incoming data by fully overwriting existing tables. This approach is inefficient and risky in production environments where only a small subset of records changes with each batch. Delta Lake's MERGE operation solves this by applying update and insert logic in a single atomic statement, using a match key to determine whether an incoming record already exists.

13.2 MERGE Implementation

```

from      delta.tables      import      DeltaTable

gold_table      =      DeltaTable.forName(spark,
"fraud_db.gold_fraud_transactions")

gold_table.alias("target").merge(
    new_transactions.alias("source"),
    "target.txn_id      =      source.txn_id"
).whenMatchedUpdateAll() \
    .whenNotMatchedInsertAll() \
    .execute()

```

13.3 MERGE Logic Explained

Condition	Action / Result
target.txn_id = source.txn_id (match found)	WHEN MATCHED UPDATE SET * — Existing row is updated with new values
target.txn_id not in source (no match)	WHEN NOT MATCHED INSERT * — New row is inserted into the Gold table

13.4 Advantages of Delta MERGE Observed

- ACID compliance: the entire MERGE either completes fully or is rolled back — no partial writes.
- Single statement handles both updates and inserts — no need for separate jobs.
- Schema enforcement prevents mismatched columns from corrupting the table.
- Transaction log records every MERGE as a versioned entry, enabling full auditability.
- No full table rewrite — only affected rows are modified, improving performance at scale.

14. Results

The pipeline produces the following output metrics upon successful execution. Actual numerical values are generated from Notebook 5 (Fraud Analytics) and the Databricks notebook.

Metric	Description
Total Transactions	Complete count of all processed transactions in the Gold layer
Total Fraud Transactions	Count of transactions where fraud_flag = "fraud"
Total Normal Transactions	Count of transactions where fraud_flag = "normal"
Fraud Rate (%)	Percentage of total transactions classified as fraudulent
Total Fraud Amount	Sum of transaction amounts for all flagged fraud records
Fraud by Account	Breakdown of fraudulent transactions grouped by account_id
Fraud by Type	Count of fraud transactions grouped by fraud_type from watchlist
Fraud by Merchant	Count of fraud transactions grouped by merchant name

Metric	Description
High-Risk Accounts	Accounts with multiple fraud transactions ordered by fraud_amount
Final Row Count after MERGE	Total rows in Gold table after incremental MERGE execution

15. Visualisations

The following visualisations are generated in Notebook 5 using Matplotlib and exported to the visualizations/ directory:

Chart	Description
Fraud vs Normal Transactions	Bar chart comparing total fraud and normal transaction counts
Fraud by Type	Bar chart showing volume of each fraud category from the watchlist
Top Fraudulent Accounts	Horizontal bar chart of top 10 accounts by fraud transaction count
Risk Distribution	Distribution of transaction amounts across risk categories
Fraud by Merchant	Bar chart showing merchants most frequently associated with fraud

16. Project Notebooks

Notebook	Purpose
01_Data_Analysis.ipynb	Load datasets, explore schema, identify data quality issues
02_Bronze_Layer_Ingestion.ipynb	Read raw CSV files, create Bronze DataFrames, preserve source data
03_Silver_Layer_Cleaning.ipynb	Clean data, remove nulls/duplicates, cast types, join tables
04_Gold_Fraud_Detection.ipynb	Match transactions with watchlist, apply fraud_flag classification
05_Fraud_Analytics_and_Visualization.ipynb	Run Spark SQL queries, generate charts and business insights

Notebook	Purpose
06_Databricks_Delta_Implementation.ipynb	Replicate full pipeline in Azure Databricks, create Delta tables, run MERGE

17. Project Folder Structure

```

Smart_Fraud_Detection_Pipeline/
|
+-- Dataset/
|   +-- raw/
|       +-- accounts.csv
|       +-- fraud_watchlist.csv
|       +-- transactions.csv
|
|   +-- bronze/
|       +-- bronze_accounts.csv
|       +-- bronze_fraud_watchlist.csv
|       +-- bronze_transactions.csv
|
|   +-- silver/
|       +-- silver_accounts.csv
|       +-- silver_enriched_transactions.csv
|       +-- silver_fraud_watchlist.csv
|       +-- silver_transactions.csv
|
|   +-- gold/
|       +-- gold_accounts.csv
|       +-- gold_fraud_watchlist.csv
|       +-- gold_transactions.csv
|
+-- notebook_implementation/
|   +-- Databricks_notebook/
|       +-- 01_Bronze_Ingestion.ipynb
|       +-- 02_Silver_Delta.ipynb
|       +-- 03_Gold_Fraud_Features.ipynb
|       +-- 04_Fraud_Model.ipynb
|
|   +-- Jupyter_notebook/
|       +-- 01_Data_Profiling_and_Validation.ipynb
|       +-- 02_Bronze_Layer_Ingestion.ipynb
|       +-- 03_Silver_Layer_Cleaning(1).ipynb
|       +-- 04_Gold_Layer_Fraud_Detection(1).ipynb
|       +-- 05_Fraud_Analytics_and_Visualization.ipynb
|
+-- notebook_output/

```

```

|-- Databricks_Notebook_Output/
|   |-- 01_Bronze_Ingestion.pdf
|   |-- 02_Silver_Delta.pdf
|   |-- 03_Gold_Fraud_Features.pdf
|   |-- 04_Fraud_Model(1).pdf
|
|-- jupyter_notebook_output/
|   |-- 01_Data_Profiling_and_Validation.pdf
|   |-- 02_Bronze_Layer_Ingestion.pdf
|   |-- 03_Silver_Layer_Cleaning(1).pdf
|   |-- 04_Gold_Layer_Fraud_Detection(1).pdf
|   |-- 05_Fraud_Analytics_and_Visualization.pdf
|
|-- output_csv/
|   |-- fraud_by_merchant.csv
|   |-- fraud_by_type.csv
|   |-- fraud_summary.csv
|   |-- risk_summary.csv
|   |-- top_fraud_accounts.csv
|
+-- Smart_Fraud_Detection_Pipeline_Report.pdf
+-- README.md

```

18. Data Engineering Concepts Demonstrated

Concept	Application in this Project
ETL / ELT Pipeline	Full pipeline from raw CSV ingestion through cleansed and business-ready output
Medallion Architecture	Bronze / Silver / Gold separation of data quality concerns
PySpark	All data processing performed using distributed Spark DataFrames
Spark SQL	Analytical views and aggregations on Gold Delta tables
Data Cleaning	Null handling, deduplication, type casting, and whitespace trimming in Silver layer
Data Validation	assert-based checks for nulls, duplicates, and row counts
Schema Enforcement	Delta Lake schema enforcement prevents malformed records at write time

Concept	Application in this Project
Data Enrichment	Transactions enriched with account information via inner join
Delta Lake	All layers stored as Delta tables with ACID compliance
MERGE / Upsert	Incremental processing using DeltaTable.merge() without full rewrites
Azure Databricks	Cloud-based pipeline execution on a managed Spark cluster
Business Analytics	Fraud metrics, account-level analysis, and merchant-level summaries
Data Visualization	Matplotlib charts for fraud trends and risk distribution

19. Future Scope

The current pipeline can be extended in the following directions to enhance its capabilities and bring it closer to a production deployment:

- Implement machine-learning-based fraud prediction using PySpark MLlib or scikit-learn.
- Add real-time fraud detection using Spark Structured Streaming to process live transaction feeds.
- Connect the Gold Delta tables to Power BI or Tableau for interactive executive dashboards.
- Automate the full pipeline using Azure Data Factory or Databricks Workflows with scheduled triggers.
- Add automated data quality monitoring with alerting using Great Expectations or Databricks DLT.
- Implement advanced risk scoring models combining transaction amount, frequency, and merchant patterns.
- Integrate external fraud intelligence feeds to enrich the watchlist with third-party risk signals.

20. Conclusion

The Smart Fraud Detection Pipeline demonstrates a complete, end-to-end data engineering solution for processing and analysing financial transaction data. Starting from raw CSV files containing realistic data quality issues, the pipeline applies the Medallion Architecture to progressively clean, enrich, and transform data through Bronze, Silver, and Gold layers. PySpark

is used for all distributed data processing, while Spark SQL generates actionable business insights from the final Gold tables.

The second phase of the project successfully extends the pipeline to Azure Databricks, where all three layers are stored as Delta tables. Delta Lake provides ACID-compliant storage, schema enforcement, full transaction history, and the ability to perform incremental data processing through the MERGE operation. The MERGE implementation ensures that new or modified records can be incorporated into the Gold table without rewriting existing data, closely reflecting how real-world data engineering pipelines handle continuously arriving transaction streams.

Overall, this project demonstrates how modern data engineering tools and architectural patterns can be combined to build a reliable, scalable, and maintainable fraud detection pipeline. The skills applied — including distributed processing, Delta Lake, Medallion Architecture, Spark SQL analytics, and cloud-based execution on Azure Databricks — reflect the core competencies of a production data engineering role.

Snehal A. Bhosale

Sanjivani College of Engineering, Kopergaon

Celebal Technologies – Data Engineering Internship

CEI ID: CT_CSI_DE_1177

snehalbhosale1807@gmail.com